SC4052 Assignment 2

Using GraphRAG for knowledge graph-based information retrieval.

Gary Quah YiKai <U2322511D>

Table of contents:

6ii) Challenge question: GraphRAG **2**
- Introduction to GraphRAG 2
- Single-hop queries vs multi-hop queries 2
- Exploring GraphRAG with multi-hop queries. 2
- Summary of GraphRAG vs Google's random surfer pagerank 3
- Software Implementation 3
  - Stack 3
  - Data handling 3
  - Graph creation 4
  - Retrieving pagerank score 4
  - Output analysis 6

References **8**

6ii) Challenge question: GraphRAG

Introduction to GraphRAG

RAG stands for Retrieval Augmented Generation, allowing Artificial Intelligence (AI) to ground its information retrieved based on actual data, such as from a large dataset.

GraphRAG uses graphs such as knowledge graphs to represent and store the data, making information retrieval faster and more accurate. The data is typically represented in the form of triples containing subject, relation and object.

Knowledge graphs store information by representing entities or objects as nodes and having relations define the links between nodes. For example, for the statement “Marie Curie is a woman”, “Marie Curie” and “Woman” are entities while “is a” represents the relationship between the two nodes.

Single-hop queries vs multi-hop queries

Single-hop queries refer to directly fetching information related to the query. For example, if someone asks “Who is Marie Curie?”, it would likely return “Marie Curie is a scientist”, returning only data from a fact or node.

In comparison, multi-hop queries would navigate through multiple nodes. If someone asks the same question, “Who is Marie Curie?”, the answer would reply with data from more than one fact or node, making the answer to the query more complete and informative, for example “, Marie Curie is a scientist who is also the first woman to win a Nobel prize.”

Exploring GraphRAG with multi-hop queries.

For graphRAG, on top of the pagerank algorithm, a personalisation vector is also included to create bias jumps towards the query nodes. Rather than jumping to random nodes in the graph, graphRAG pagerank would jump to the query nodes with equal probability and continue down the graph, causing convergence on the query nodes, giving a higher pagerank score to nodes that are closer to the query nodes.

GrapRAG leverages the ability of Natural Language Processing (NLP) to identify key terms in a query, and runs GraphRAG with these key terms to identify other nodes with the highest relevance to the query. These retrieved nodes are then passed to a Language Learning Model (LLM) to generate a natural language answer pertaining to the query.

Doing so allows Artificial Intelligence (AI) to ground its knowledge on facts, reducing the chances of hallucination.

Summary of GraphRAG vs Google's random surfer pagerank

Comparing GraphRAG-based pagerank to traditional pagerank used by Google, which would instead jump to a random node rather than the query node, this allows GraphRAG-based pagerank scoring to converge on nodes with the highest importance related to the query nodes and their connected neighbours, as they would tend to be visited more often, giving them a higher pagerank score.

Software Implementation

Stack

To explore the usage of GraphRAG using knowledge graphs to find nodes of highest relevancy to query nodes, a software implementation using Python, NetworkX, and the matplotlib library will be created to explore, understand and demonstrate GraphRAG.

Data handling

A small data sample has been created in a text file in the form of triples, representing node and edge structures in a knowledge graph in the order of subject, relation and object.

```
Marie Curie,discovered,radium
Marie Curie,discovered,polonium
radium,used in,radiotherapy
polonium,used in,cancer treatment
radiotherapy,advances,medical imaging
cancer treatment,advances,medical imaging
radiation safety,related to,medical imaging
Marie Curie,won,Nobel Prize
Nobel Prize,recognized in,physics
physics,influenced,medical imaging
```

The graph was then created using the NetworkX library and rendered out using the matplotlib library. As the data in the text file is now in the form of triples rather than edges, processing is needed in order for a graph to be created using the NetworkX library.

The lines in the file are stripped of trailing white spaces and split by commas. The subject and object (triples) are saved as edges representing nodes, and the relations are stored in a dictionary with the edges as the key for quick lookup.

```python
script_dir = os.path.dirname(os.path.abspath(__file__))
triples_file_path = os.path.join(script_dir, "triples.txt")

edges = []
relations = {}

with open(triples_file_path, 'r', encoding='utf-8') as f:
    for line in f:
        parts = line.strip().split(',')
        if len(parts) == 3:
            subject = parts[0].strip()
            relation = parts[1].strip()
            obj = parts[2].strip()
            edges.append((subject, obj))
            relations[(subject, obj)] = relation
```

Graph creation

We create a graph using the NetworkX library, using edges extracted from triples.txt when we read the data at the start.

We keep relations separate, with the edge storing the subject and object as the key to access the relation.

```
Edges: [('Marie Curie', 'radium'), ('Marie Curie', 'polonium'), ('radium', 'radiotherapy'), ('polonium', 'cancer treatment'), (
'radiotherapy', 'medical imaging'), ('cancer treatment', 'medical imaging'), ('radiation safety', 'medical imaging'), ('Marie C
urie', 'Nobel Prize'), ('Nobel Prize', 'physics'), ('physics', 'medical imaging')]

Relations: {('Marie Curie', 'radium'): 'discovered', ('Marie Curie', 'polonium'): 'discovered', ('radium', 'radiotherapy'): 'us
ed in', ('polonium', 'cancer treatment'): 'used in', ('radiotherapy', 'medical imaging'): 'advances', ('cancer treatment', 'med
ical imaging'): 'advances', ('radiation safety', 'medical imaging'): 'related to', ('Marie Curie', 'Nobel Prize'): 'won', ('Nob
el Prize', 'physics'): 'recognized in', ('physics', 'medical imaging'): 'influenced'}
```

```python
G = nx.DiGraph()
G.add_edges_from(edges)
```

Retrieving pagerank score

We use the built-in pagerank implementation from the NetworkX library, using nx.pagerank and create a personalisation vector where each node in the graph is assigned a count equal to how many keywords the node contains. Nodes that have no keywords are assigned a score of 0, indicating no bias towards the random teleportation.

For a personalised pagerank, the total personalisation score must sum to 1, and thus the personalisation vector is normalised.

```python
personalization = {}

# Count keyword matches for each node
for node in G.nodes():
    count = 0
    node_lower = node.lower()
    for k in keywords:
        if k in node_lower:
            count += 1
    personalization[node] = count

# Normalize to sum to 1
total = sum(personalization.values())
for node in personalization:
    personalization[node] = personalization[node] / total
```

After running the pagerank algorithm, we need to create a new result to add relations back into the triple, as the graph only contains edges (subject and object).

```python
alpha = 0.85
pr_scores = nx.pagerank(G, alpha=alpha, personalization=personalization, tol=1e-6)

top_k = 5
top_nodes = sorted(pr_scores.items(), key=lambda x: x[1], reverse=True)[:top_k]

print("\nTop relevant nodes:")
for node, score in top_nodes:
    print(f"{node} : {score:.5f}")

result = []

# Add the relations with the top nodes
for node, _ in top_nodes:
    for obj in G.successors(node):
        if (node, obj) in relations:
            relation = relations[(node, obj)]
            result.append((node, relation, obj))

for subject, relation, object in result:
    print(f"{subject},{relation},{object}")
```

Output analysis

Simulating an NLP extracting out keywords, the keyphrase of “Marie Curie and Radium” is passed into the algorithm, resulting in “Marie Curie” and “Radium” being passed as the seed nodes into the software implementation.

After printing out the top relevant nodes with the highest pagerank score, the following are the nodes with the highest pagerank score.

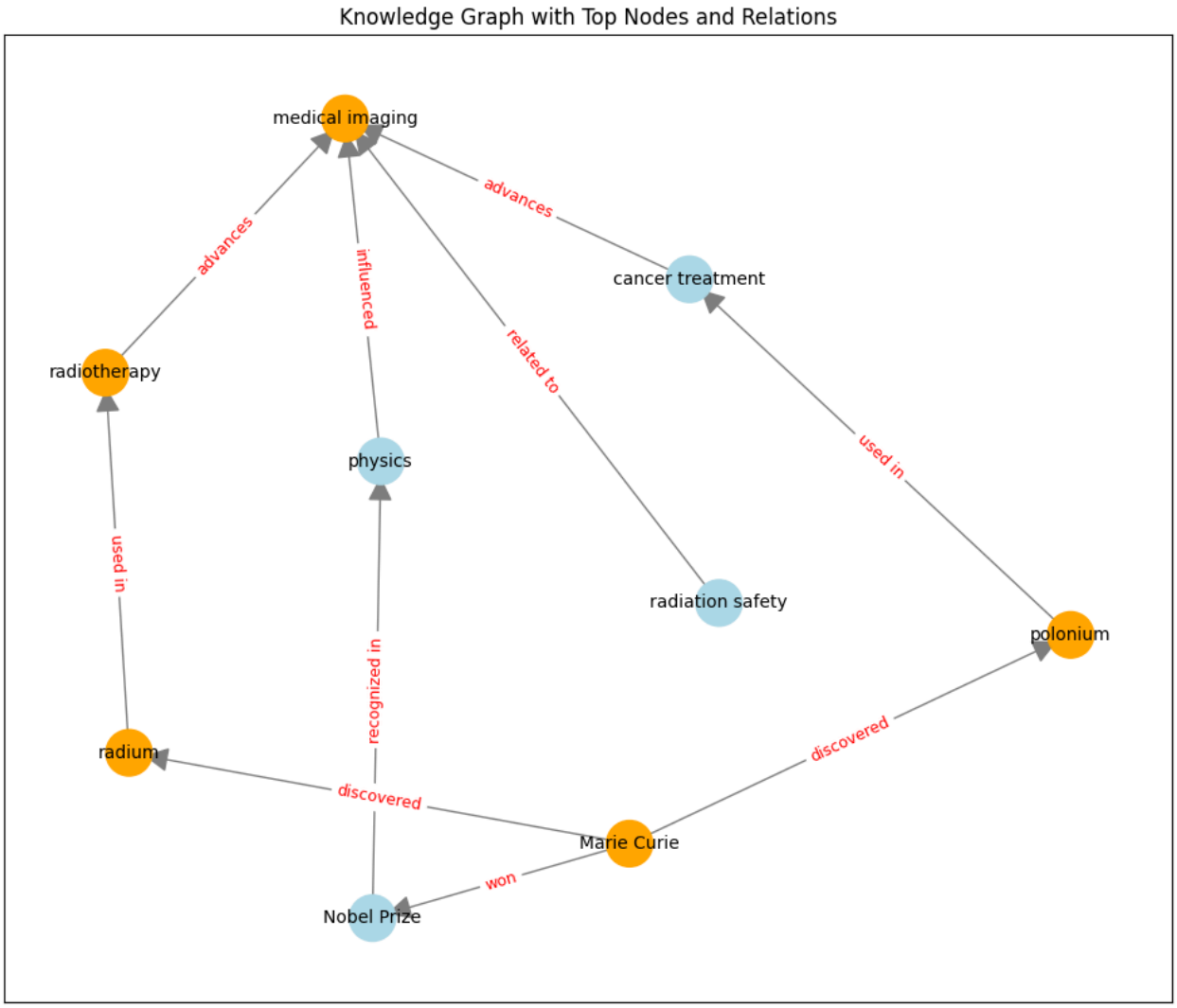
Mentioned previously in the creation of a graph using the NetworkX library, we stored the relations as a dictionary using the edges as a key to access the relation.

```
# Add the relations with the top nodes
for node, _ in top_nodes:
    for obj in G.successors(node):
        if (node, obj) in relations:
            relation = relations[(node, obj)]
            result.append((node, relation, obj))

for subject, relation, object in result:
    print(f"{subject},{relation},{object}")
```

```
Top relevant nodes:
Marie Curie : 0.22357
medical imaging : 0.21807
radium : 0.17513
radiotherapy : 0.14886
polonium : 0.06334
Marie Curie,discovered,radium
Marie Curie,discovered,polonium
Marie Curie,won,Nobel Prize
radium,used in,radiotherapy
radiotherapy,advances,medical imaging
polonium,used in,cancer treatment
```

The nodes with the highest pagerank score would then be passed to an LLM to process and return a suitable response to the query requested from the AI, allowing data to be grounded, avoiding the hallucination of data.



From the graph and results above, it is expected that “Radium” and “Marie Curie” as the main seed nodes, are part of the highest page rank scores result. Nodes close to the seed node would result in the pagerank score converging near the seed nodes, as seen from “Radiotherapy”.

Additionally, we can also see “Medical imaging” having a high pagerank score, as a result of multiple branches eventually converging on medical imaging, giving it a higher pagerank score.

Code: https://github.com/GaryQuah/SC4052-Cloud-computing/tree/main/Assignment%202

References

1. NetworkX, “NetworkX — NetworkX documentation,” *Networkx.org*, 2025. https://networkx.org/en/
2. Enzo, “What Is GraphRAG? - Graph Database & Analytics,” *Graph Database & Analytics*, Dec. 05, 2024. https://neo4j.com/blog/genai/what-is-graphrag/
3. Shesh6, “CS166--Modeling-Simulation-and-Decision-Making/Homework 82 - PageRank as a Random Walk.ipynb at master · Shesh6/CS166--Modeling-Simulation-and-Decision-Making,” *GitHub*, 2025. https://github.com/Shesh6/CS166--Modeling-Simulation-and-Decision-Making/blob/master/Homework%2082%20-%20PageRank%20as%20a%20Random%20Walk.ipynb